iris

First-Class Multi-GPU Programming Experience in Triton



Research and advanced Development (RAD)
AMD

AMD Triton Team

Public

Agenda

01 Programming with Iris

02 CU- and SDMA-based collectives

03 Inference with Iris

04 What’s next



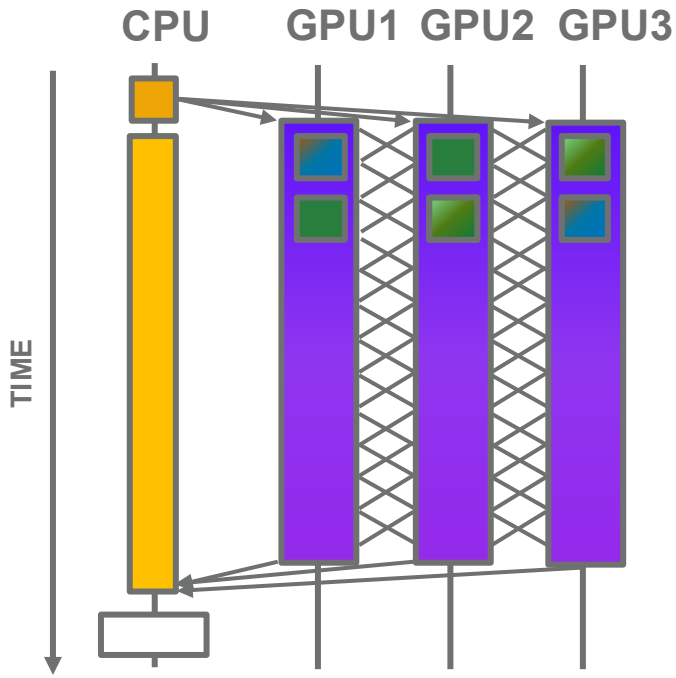
iris

AMD's open-source Triton† -based framework for Remote Memory Access.

- Make Multi-GPU programming a first-class citizen in Triton while retaining its programmability and performance
- Overlap communication with compute tile by tile in a single kernel, written natively in Triton

```
import iris
```

Github.com/ROCm/iris



† Iris offers both Triton and Gluon APIs

Public

Programming with Iris



Host-Side Iris APIs

```python
import triton
import iris

# Initialize Iris symmetric heap (1 GiB)
ctx = iris.iris(1 << 30)
heap_bases = ctx.get_heap_bases()


# Allocate tensors on the symmetric heap
n = 4096
buffer = ctx.zeros(n, dtype=iris.float32, device="cuda")


# Launch a Triton kernel that uses Iris RMA
BLOCK = 1024
grid = (triton.cdiv(n, BLOCK),)
kernel[grid](buffer, n, BLOCK, heap_bases)


# Synchronize across Iris ranks
ctx.barrier()
```

And more PyTorch-Like tensor creation APIs!

```python
iris.full(…)
iris.ones(…)
iris.zeros(…)
iris.zeros_like(…)

iris.arange(…)
iris.linspace(…)

iris.empty(…)
iris.randint(…)
iris.rand(…)
iris.randn(…)
```



Simple load & store APIs

```python
import triton
import triton.language as tl
import iris

# Host-side setup
host_ctx = iris.iris(heap_size=2**30)
context_tensor = host_ctx.get_device_context()

@triton.jit
def kernel(buffer, buffer_size: tl.constexpr,
        block_size: tl.constexpr, context_tensor):

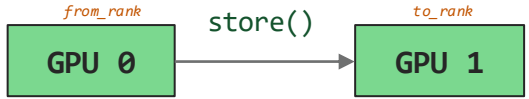
    # Initialize device-side Iris context
    ctx = DeviceContext.initialize(context_tensor)

    # Compute global offsets
    pid = tl.program_id(0)
    offsets = pid * block_size + tl.arange(0, block_size)
    mask = offsets < buffer_size

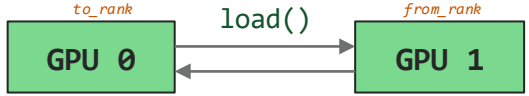
    # Remote write to GPU 1
    ctx.store(buffer + offsets,
            1,      # value to write
            to_rank=1,
            mask=mask)
```

```python
# Aggregate-based device-side context
class IrisDeviceContext:
    @triton.jit
    def initialize(context_tensor):
        """Initialize device context."""
        ...

    @triton.jit
    def store(self, pointer, value, to_rank,
              mask=None):
        """Write data to the specified rank."""
        ...
```




```python
    @triton.jit
    def load(self, pointer, from_rank, mask=None):
        """Load data from the specified rank.
        Returns: Block containing the loaded value.
        """
        ...
```



It's Triton all the way down

Host APIs Device APIs

ctx.ops

Compute fused with communication

ctx.ops.matmul_all_reduce(output, A, B)

all_gather_matmul matmul_all_gather matmul_reduce_scatter

ctx.ccl

Collectives

ctx.ccl.all_reduce(output, input)

all_gather reduce_scatter all_to_all

Iris RMA

RMA primitives

ctx.load(ptr, from_rank=from_rank)

ctx.store(ptr, val, to_rank=to_rank)

get put copy quiet 8 atomics

tritonBLAS†

tritonblas.kernels.stages

GemmContext Tile make_tensor_view

† github.com/ROCm/tritonBLAS

Public

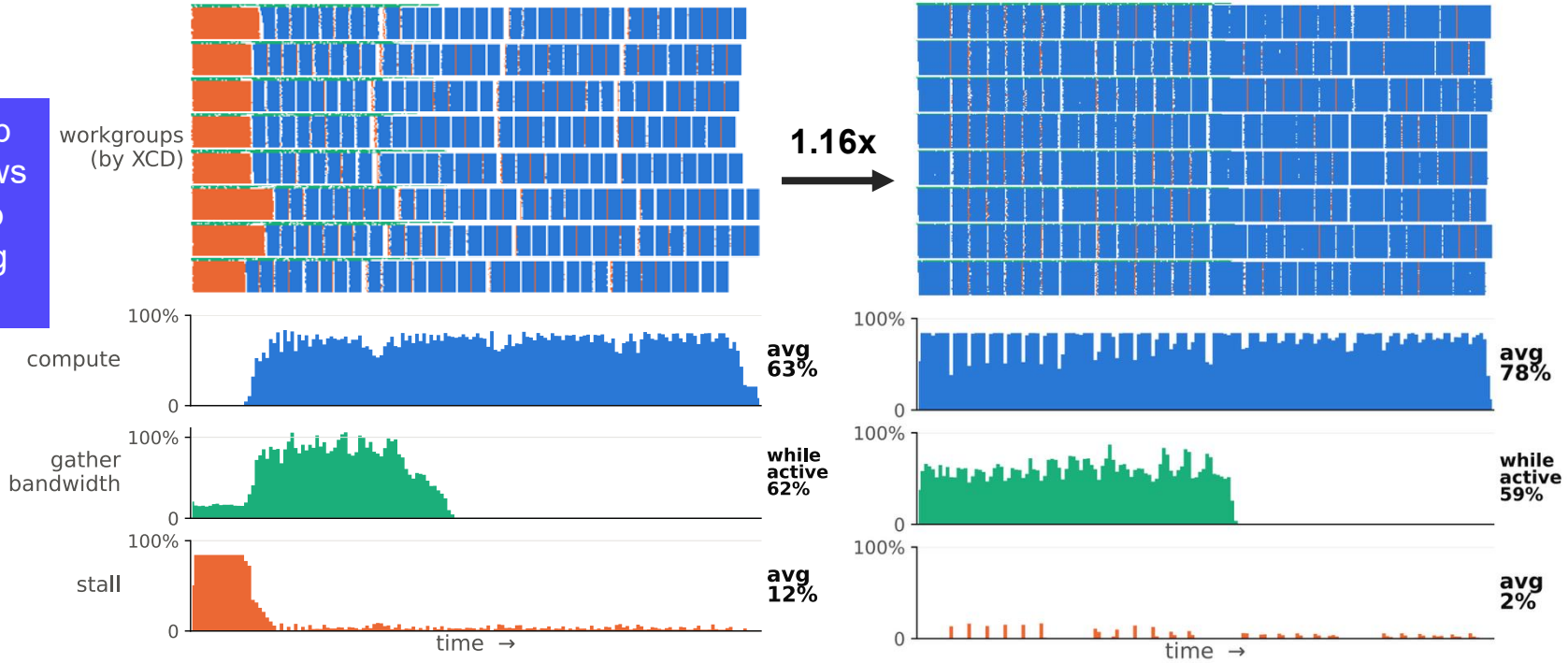
CU-driven collectives



Observability of Overlap and Dependency Stalls

Workgroup trace shows workgroup scheduling to CUs

8192x8192x8192 4 GPUs AG+MM

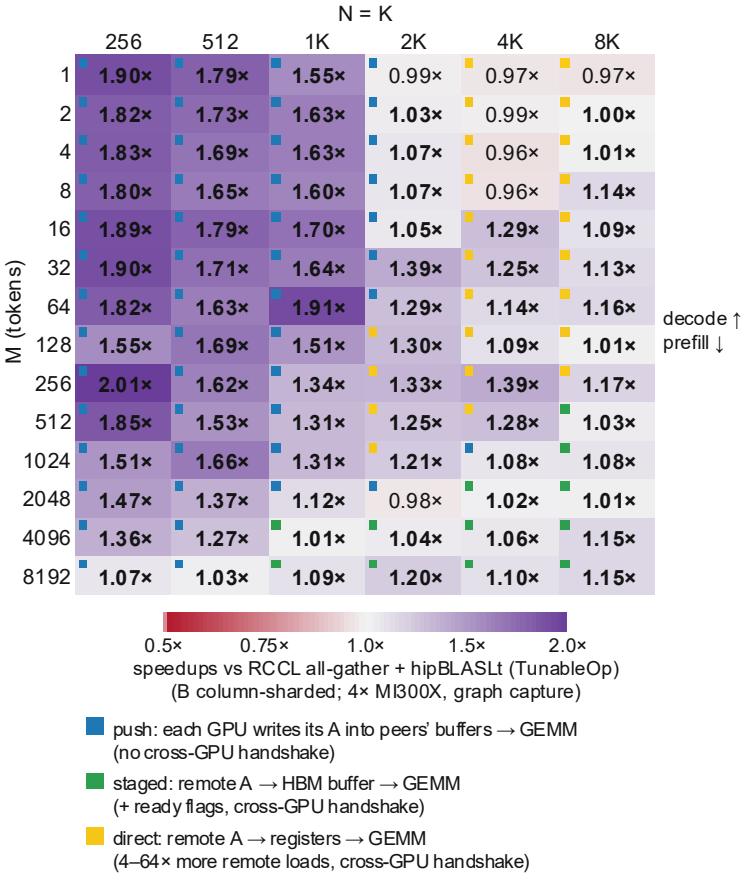


compute
stall: waiting for data
all-gather

With this workgroup trace, we can compute global dependency stall, compute utilization, and bandwidth

Performance of CU-Initiated AG+ GEMM vs RCCL+HB

- Workgroup-Specialized with HBM buffer for pipelining
- Each size requires tuning to balance pipeline between compute and communication
- CU-initiated communication is preferred when spare CUs are available (GEMM tiles < #CUs)
- Triton GEMM Slower than hipBLASLt on large compute bound sizes



*Fused kernel is tuned per size

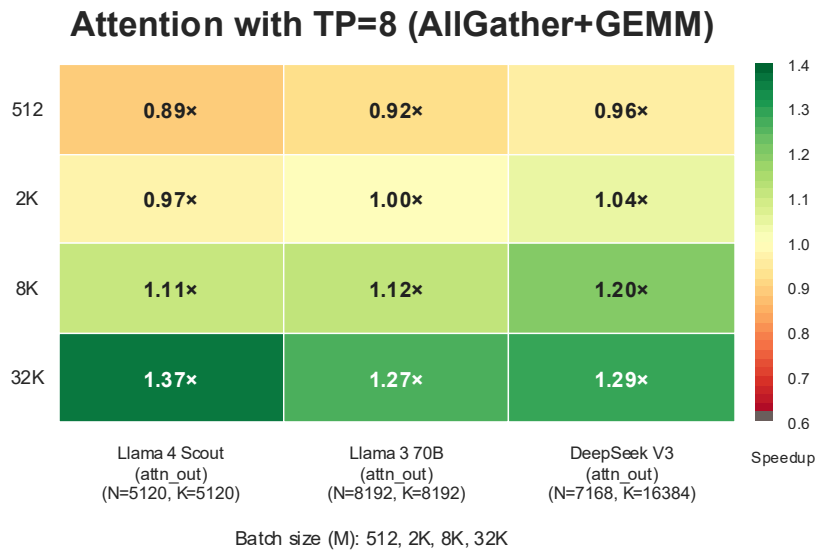
Public

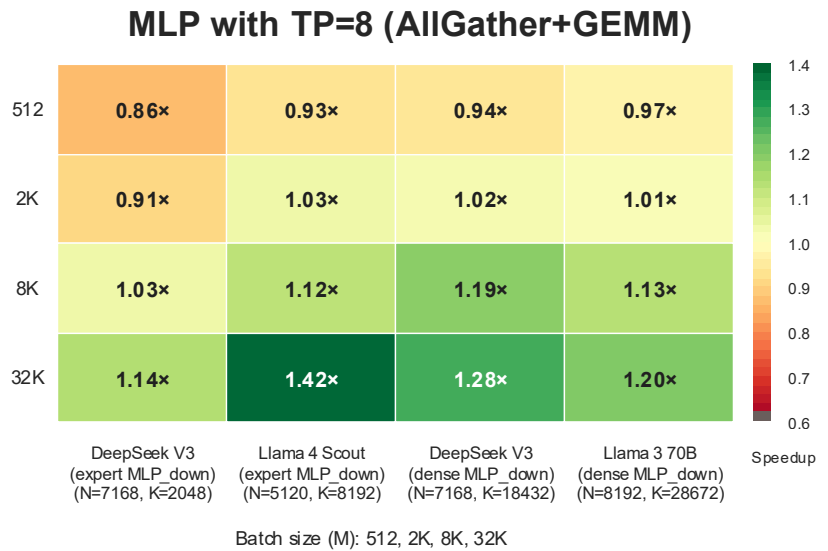
SDMA offload



Speedup with SDMA Offload

SDMA offload is beneficial in the low interactivity decode and prefill phase of LLM inference





Speedup over RCCL AllGather + tritonBLAS GEMM

Public

Inference with Iris



Iris carries every Kimi K3 collective in TokenSpeed

Protocols written in Gluon on Iris · 2 per layer × 92 layers ≈ 184 collectives per decode token

Attention › **Iris all-reduce** › **AttnRes + norm** › **MoE experts** › **Iris all-reduce**

Small · decode

One-shot push, fused with residual, AttnRes, RMSNorm

Medium

Two-shot pull

Large · prefill

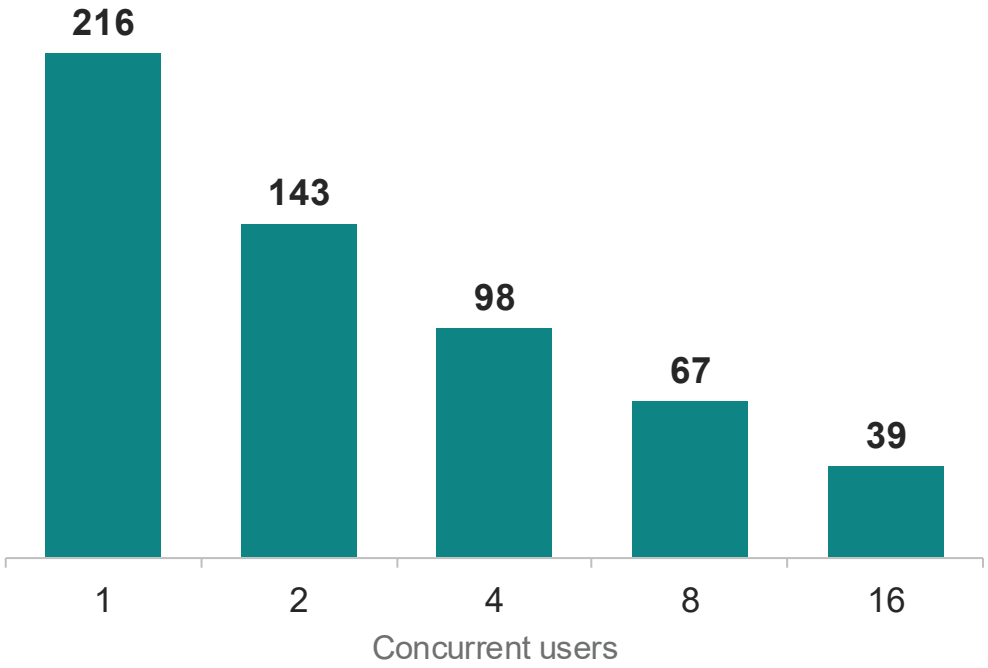
Reduce-scatter → M/8-row epilogue → all-gather

One Iris buffer for the MoE reduction: expert kernels write routed and shared partials straight into it, so one collective reduces both, with no concatenation copy.



tokenspeed@7e2f1deb · AMD technical article, Kimi K3 on MI355X with TokenSpeed, Oct 2026

Kimi K3 end to end: 216 output tokens/s per user



4.62 ms

per output token at one user, with every collective on Iris

8× MI355X · TP8 · EAGLE3 · FP8 KV · multi-turn agentic coding, ~50K-token context · tokenspeed@7e2f1deb

Multi-GPU programming as a first-class citizen in **Triton**. Communication **fused into compute**, tile by tile.




```
import iris
Github.com/ROCm/iris
https://rocm.github.io/iris
```

What's next:

- Highly optimized collectives and applications
- Novel ways to drive collectives over RDMA, SDMA and TDM
- End-to-end integration with inference and training frameworks

⭐ **202** stars | 🔱 **52** forks | 👥 **23** contributors

Open issues or reach out to:
muhaawad@amd.com

Thank you!



Public

Copyright and disclaimer

©2026 Advanced Micro Devices, Inc. All rights reserved.

AMD, the AMD Arrow logo and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate releases, for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.


AMD